

统一 ECS 错误处理 (Unifield ECS Error Handling)

1. 在拥有统一 ECS 错误处理之前

Bevy 过去一向将错误处理交由用户自行解决。虽然很多 Bevy API 会返回 Result 或错误类型，但 Bevy 的 ECS 系统和命令自身并不支持返回 Result 或错误。此外，Bevy 还提供了一些更简洁的“快速报错”API (会 panic)，用起来比显式处理 Result 更省事。这些因素共同促使开发者在错误处理上过于依赖 panic，即使他们并不希望如此。

2. 统一 ECS 错误处理介绍

理想情况下，开发者应该能够自行选择系统是否在出错时 panic。比如在开发阶段选择 panic，以便快速发现并修复错误；而在发布到生产环境时，则可以选择将错误打印到控制台，以免影响用户体验。

Bevy 0.16 引入了一个统一的错误处理范式，帮助你开发出无崩溃的游戏（或其他应用），同时保留 panic 带来的快速开发优势。

我们设计了一个简单一致的错误处理系统，并附带了一些便于调试的小功能：

- Bevy 的系统、观察者、命令现在支持返回 Result 类型，即 Result<(), BevyError> 的类型别名；
- BevyError 是一个新的通用错误类型，支持接收任意错误类型（类似 anyhow），这使得你可以在系统、观察器和命令中轻松使用？运算符捕获并返回错误；
- BevyError 会自动捕获高质量的自定义回溯信息，默认情况下会过滤掉 Rust 和 Bevy 内部的干扰信息，只保留核心堆栈，提升可读性；
- 所有系统、观察者和命令返回的错误，都会交由 GLOBAL_ERROR_HANDLER 处理。它默认行为是 panic，但你也可以自定义为记录日志、panic 或其他处理逻辑。我们建议在开发时使用默认 panic 行为，在生产环境中切换为记录错误日志；
- 现在我们鼓励开发者尽可能将错误向上传递（bubble up），而不是在错误发生处立刻 panic。

3. 使用统一 ECS 错误处理

```
// 命令统一 ECS 错误处理
fn setup(mut commands: Commands) {
    commands.queue(|world: &mut World| -> Result {
        world.spawn((Name::new("MyPlayer"), MyPlayer))
            .observe(observer_unifield_error_handling);

        Ok(())
    });
}

// 系统统一 ECS 错误处理
fn unifield_error_handling(players: Query<&Name, With<MyPlayer>>) -> Result {
    let name = players.single()?;
    info!("player name: {name}");

    Ok(())
}

// 观察者统一 ECS 错误处理
fn observer_unifield_error_handling(
    _trigger: Trigger<OnAdd>,
    players: Query<&Name, With<MyPlayer>>,
) -> Result {
    let name = players.single()?;
    info!("player name: {name}");

    Ok(())
}
```

4. 设置统一 ECS 错误处理器

想要设置统一 ECS 错误处理器，要 configurable_error_handler feature 。

Bevy 引擎提供了一系列的错误处理器：

- panic：错误将导致恐慌
- error：在 error 日志级别记录错误
- warn：在 warn 日志级别记录错误
- info：在 info 日志级别记录错误
- debug：在 debug 日志级别记录错误
- trace：在 trace 日志级别记录错误
- ignore：忽略错误

如果 Bevy 引擎提供的一系列错误处理器无法满足需要，你可以自定义统一 ECS 错误处理器。

```
// 自定义统一 ECS 错误处理器
fn my_error_handler(error: BevyError, ctx: ErrorContext) {
    if ctx.name().ends_with("unifield_error_handling") {
        info!("{ } 函数出错了", ctx.name());
        return;
    }

    // 在 error 日志级别记录错误
    ecs::error::error(error, ctx);
}

// 设置统一 ECS 错误处理器
GLOBAL_ERROR_HANDLER
    .set(my_error_handler)
    .expect("只能设置一次");
```

5. 单独错误处理

统一 ECS 错误处理无法满足所有需求，你可以通过管道为部分系统使用单独的错误处理。

```
// 单独错误处理
fn separate_error_handling(players: Query<&Name, With<MyPlayer>>) -> Result<(),
QuerySingleError> {
    let name = players.single()?;
    info!("palyer name: {name}");

    Ok(())
}

// 单独错误处理器
fn separate_error_handler(In(result): In<Result<(), QuerySingleError>>) {
    if let Err(error) = result {
        info!("error: {error}");
    }
}

App::new()
    .add_systems(Update, separate_error_handling.pipe(separate_error_handler))
    .run();
```